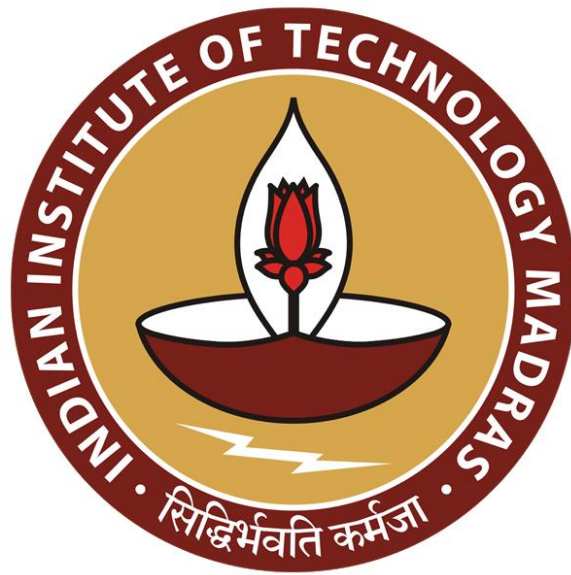




IIT Madras
ONLINE DEGREE

Programming Concepts Using Java
Professor Madhavan Mukund
Department of Computer Science
Chemical Mathematical Institute
Indian Institute of Technology, Madras
Private Classes

(Refer Slide Time: 00:14)



Private classes

Madhavan Mukund

<https://www.cmi.ac.in/~madhavan>

Programming Concepts using Java

Week 4



Nested objects

- An instance variable can be a user defined type
 - `Employee` uses `Date`

```
public class Employee{  
    private String name;  
    private double salary;  
    private Date joindate;  
    ...  
}
```

```
public class Date {  
    private int day, month year;  
    ...  
}
```



Madhavan Mukund

Private classes

Programming Concepts using Java 2/4

- An instance variable can be a user defined type
 - `Employee` uses `Date`
- `Date` is a public class, also available to other classes
- When could a private class make sense?

```
public class Employee{
    private String name;
    private double salary;
    private Date joindate;
    ...
}

public class Date {
    private int day, month year;
    ...
}
```



Mohanan Mithund

Private classes

Programming Concepts using Java 2/4

So, let us look at something which does not seem normal at first sight, a private class. So, we have seen that we can have objects which contain other objects in their instance variables. For instance, when we create an employee, we might want to record the joining date of the employee, and the joining date can be recorded in terms of another class called date. So, we have a public class called date, which has its own instance variables its own structure, date, day, month, and year, and this is a nested object which is sitting inside an employee object.

So, in this case, date is a public class and it is also available to other classes. So, a employee uses date, but there is nothing to prevent other classes from using date. And this is then a public class available throughout the system. So, what we are going to discuss now is private classes. So, why would you need a private class and when would it make sense.

(Refer Slide Time: 01:17)

- `LinkedList` is built using `Node`

```
public class Node {
    public Object data;
    public Node next;
    ...
}

public class LinkedList {
    private int size;
    private Node first;

    public Object head() {
        Object returnval = null;
        if (first != null) {
            returnval = first.data;
            first = first.next;
        }
        return returnval;
    }
}
```



Madhavan Mukund

Private classes

Programming Concepts using Java 3/4

- `LinkedList` is built using `Node`
- Why should `Node` be public?
 - May want to enhance with `prev` field, doubly linked list
 - Does not affect interface of `LinkedList`

```
public class Node {
    public Object data;
    public Node next;
    ...
}

public class LinkedList {
    private int size;
    private Node first;

    public Object head() {
        Object returnval = null;
        if (first != null) {
            returnval = first.data;
            first = first.next;
        }
        return returnval;
    }
}
```



Madhavan Mukund

Private classes

Programming Concepts using Java 3/4

- `LinkedList` is built using `Node`
- Why should `Node` be public?
 - May want to enhance with `prev` field, doubly linked list
 - Does not affect interface of `LinkedList`
- Instead, make `Node` a private class
 - Nested within `LinkedList`
 - Also called an `inner class`

```
public class LinkedList {
    private int size;
    private Node first;

    public Object head() { ... }

    public void insert(Object newdata) {
        ...
    }

    private class Node {
        public Object data;
        public Node next;
        ...
    }
}
```



Madhavan Mukund

Private classes

Programming Concepts using Java 3/4

- `LinkedList` is built using `Node`
- Why should `Node` be public?
 - May want to enhance with `prev` field, doubly linked list
 - Does not affect interface of `LinkedList`
- Instead, make `Node` a private class
 - Nested within `LinkedList`
 - Also called an inner class
 - Objects of private class can see private components of enclosing class

```
public class LinkedList{
    private int size;
    private Node first;

    public Object head(){ ... }

    public void insert(Object newdata){
        ...
    }

    private class Node {
        public Object data;
        public Node next;
        ...
    }
}
```

d.equal(02)



Madhavan Mahand

Private classes

Programming Concepts using Java 3/4

So, consider a `LinkedList`. So typically, a `LinkedList` is made up of `Nodes`. So, you first create a `Node` class. So, a `Node` class typically has two fields, so remember a `LinkedList`, typically, the picture is like this. So, you have a head pointing to the first element and then each one has a next, pointing to the next element.

So, each, each `Node` in `LinkedList` typically has a value and an x point. So, you first define a `Node` class. And then in the `LinkedList`, you say that you have maybe an extra variable to record the size of the list, and you have a `Node` pointer, in this case, first, which point to the first `Node` in the list.

And now if you want to get the value at the head of this list, for instance, this returns the head of the list and extracts it, it is like the equivalent of a pop in a stack. So, what you do is, you remove the first data item, return it, and you move the first item point to the next one. So, this is a typical behavior of `LinkedList`.

Now, in this case, `LinkedList` is using `Node` as a nested object just in the way that employee was using date. However, perhaps here there is some justification for saying that this `Node` object, or this `Node` class should not be public. Because it could very well be that we also want to add a previous, so we could have a doubly `LinkedList`.

So, we could have a list in which we point to this, the `Node` in front and the `Node` behind. And this makes it sometimes much easier to navigate a list because when you are trying to delete, for instance, in a list, you have to be a bit careful about looking one step ahead, but

you have a doubly linked list, it is much easier to delete the Node on which you are currently sitting.

Now, from the user's perspective, they are dealing with a list of values. They should not care or know whether the nodes that underlie this list have two pointers one pointer or what the structure of the pointer is. So, the structure of this Node is not something that is desirable to expose to the user of this class LinkedList because that is really an implementation detail. So, in the interest of encapsulation, we should actually not expose that.

So, this is where now private classes come in. So, what you can do is, you can push that Node class which is outside into the definition of LinkedList and make it a private class. So, now this private class has exactly the same structure as we had before. So, it has, so perhaps this could also be private. So, it has now a Node and a next, but this data value and the next value are now accessible only from inside because nobody outside even knows the existence of this Node class. So, because it is defined inside another class is sometimes also called an inner class.

So, one thing to remember is that we said that when we have a private instance variable inside a class, all objects of that class can see the private variables of each other. So, for instance, if we want to compare two objects. If you want to say `o1.equal(o2)` and do this comparison based on this instance variables, then it is impossible to do it unless `o1` can look at the values of `o2`. So, for this reason, we said that a private variable is not private within a class it is only private outside that class.

So, in the same way, this private thing is of course accessible here, but conversely, this is also accessible here. So, this private class can see the other private instance variables that are defined inside LinkedList. So, for instance, you could use the value of size inside this class it is perfectly legitimate. And we will see that this is important that we need private classes to be to actually make exploit private classes to the fullest, we will need private classes to be able to access other parts of the object inside which they reside.

So, the important thing is that they are private. Since they are private, if they are outside, then nobody can see it including this class. So, they cannot be outside, they must be inside, so they are inner classes. So, you have to define it inside the public class, which is good to use it, and

then it is accessible to that class, and whatever the class has is also accessible to that inner class.

(Refer Slide Time: 05:34)



Summary

- An object can have nested objects as instance variables
- In some situations, the structure of these nested objects need not be exposed
- Private classes allow an additional degree of data encapsulation
- Combine private classes with interfaces to provide controlled access to the state of an object



So, an object can have nested objects, this is clear, but what we have seen is that in some situations the structure of these nested objects should not be exposed. Not even need not be exposed, it probably should not be exposed. So, you can vary that extra object inside the public object as a private class and get an extra degree of data encapsulation.

So, this usage of private classes is of course, a very minimal utility. And what we are going to see is that you can do something much more fundamental private classes. You can combine private classes with the idea of an interface to allow controlled access to the state of an object. So, you remember that we access the state of an object through public methods, but sometimes you need some control over that and that control can be done through these objects which are created as private classes.